Table 1: Statistics of the preprocessed datasets. "Avg. $|\mathcal{H}|$" denotes the average length of historical interactions. "Avg. $|t_i|$" denotes the average number of tokens in the item text.

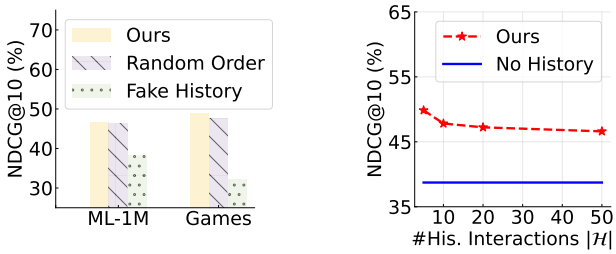
Dataset	#Users	#Items	#Interactions	Sparsity	Avg. $\|\mathcal{H}\|$	Avg. $\|t_i\|$
ML-1M	6,040	3,706	1,000,209	95.53%	46.19	16.96
Games	50,547	16,859	389,718	99.95%	7.02	43.31

the LLM outputs and the text of candidate items. We also found that LLMs occasionally generate items that are not present in the candidate set. For GPT-3.5, such deviations occur in a mere 3% of cases. One can either reprocess the illegal cases or simply treat the out-of-candidate items as incorrect recommendations.

3 Empirical Studies

Datasets. The experiments are conducted on two widely-used public datasets for recommender systems: (1) the movie rating dataset *MovieLens-1M* [24] (in short, **ML-1M**) where user ratings are regarded as interactions, and (2) one category from the *Amazon Review* dataset [46] named **Games** where reviews are regarded as interactions. We filter out users and items with fewer than five interactions. Then we sort the interactions of each user by timestamp, with the oldest interactions first, to construct the corresponding historical interaction sequences. The movie/product titles are used as the descriptive text of an item. We use item titles in this study for two reasons: (1) to determine if LLMs can make recommendations based on their intrinsic world knowledge with minimal information provided, and (2) to conserve computational resources. Exploring how LLMs use more extensive textual features for recommendations will be the focus of our future work.

Evaluation and implementation details. Following existing works [33,30], we apply the leave-one-out strategy for evaluation. For each historical interaction sequence, the last item is used as the ground-truth item in test set. The item before the last one is used in the validation set (used for training baseline methods). We adopt the widely used metric NDCG@K (in short, N@K) to evaluate the ranking results over the given m candidates, where $K \leq m$. To ease the reproduction of this work, our experiments are conducted using a popular open-source recommendation library RecBole [78]. The historical interaction sequences are truncated within a length of 50. We evaluate LLM-based methods on all users in ML-1M dataset and randomly sampled $6,000$ users for Games dataset by default. Unless specified, the evaluated LLM is accessed by calling OpenAI's API `gpt-3.5-turbo`. The hyperparameter temperature of calling LLMs is set to 0.2. *All the reported results are the average of at least three repeat runs to reduce the effect of randomness.*



(a) Perceive Order (b) Performance vs. $|\mathcal{H}|$

Fig. 2: Analysis of whether LLMs perceive the order of historical interactions.

3.1 Can LLMs Understand Prompts that Involve Sequential Historical User Behaviors?

In LLM-based methods, historical interactions are naturally arranged in an ordered sequence. By designing different configurations of $\mathcal{H}$, we aim to examine whether LLMs can leverage these historical user behaviors and perceive the sequential nature for making accurate recommendations.

LLMs struggle to perceive the order of given historical user behaviors. In this section, we examine whether LLMs can understand prompts with ordered historical interactions and give personalized recommendations. The task is to rank a candidate set of 20 items, containing one ground-truth item and 19 randomly sampled negatives. By analyzing historical behaviors, items of interest should be ranked at a higher position. We compare the ranking results of three LLM-based methods: (a) *Ours*, which ranks as we have described in Section 2.2. Historical user behaviors are encoded into prompts using the "sequential prompting" strategy. (b) *Random Order*, where the historical user behaviors will be randomly shuffled before being fed to the model, and (c) *Fake History*, where we replace all the items in original historical behaviors with randomly sampled items as fake historical behaviors. From Figure 2(a), we can see that *Ours* has better performance than variants with fake historical behaviors. However, the performance of *Ours* and *Random Order* is similar, indicating that LLMs are not sensitive to the order of the given historical user interactions.

Moreover, in Figure 2(b), we vary the number of latest historical user behaviors ($|\mathcal{H}|$) used for constructing the prompt from 5 to 50. The results show that increasing the number of historical user behaviors does not improve, but rather negatively impacts the ranking performance. We speculate that this phenomenon is caused by the fact that LLMs have difficulty understanding the order, but consider all the historical behaviors equally. Therefore too many historical user behaviors (*e.g.,* $|\mathcal{H}| = 50$) may overwhelm LLMs and lead to a performance drop. In contrast, a relatively small $|\mathcal{H}|$ enables LLMs to concentrate on the most recently interacted items, resulting in better recommendation performance.

Table 2: Performance comparison on *randomly retrieved candidates*. Ground-truth items are included in the candidate sets. "full" denotes models that are trained on the target dataset, and "zero-shot" denotes models that are not trained on the target dataset but could be pre-trained. We highlight the best performance among zero-shot recommendation methods in **bold**.

	Method	ML-1M				Games			
		N@1	N@5	N@10	N@20	N@1	N@5	N@10	N@20
full	Pop	22.91	45.16	52.33	55.36	28.35	47.42	52.96	57.45
	BPRMF [49]	34.60	59.87	64.29	65.39	44.92	62.33	66.27	68.94
	SASRec [33]	61.39	76.39	78.89	79.79	56.90	73.19	75.92	77.14
zero-shot	BM25 [50]	4.70	12.68	17.88	33.19	13.92	28.81	34.61	44.35
	UniSRec [30]	7.37	18.80	26.67	37.93	18.95	33.99	40.71	48.42
	VQ-Rec [29]	5.98	15.48	23.74	35.85	7.28	18.28	26.21	37.62
	Sequential	18.28	36.35	42.85	49.02	30.28	45.48	50.57	56.55
	Recency-Focused	19.57	37.73	44.23	50.01	**34.03**	**48.77**	**53.50**	**59.01**
	In-Context Learning	**21.77**	**39.59**	**45.83**	**51.62**	33.95	48.44	53.10	58.92

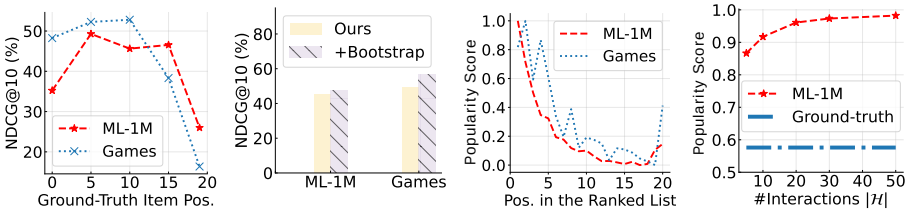
Triggering LLMs to perceive the interaction order. Based on the above observations, we find it difficult for LLMs to perceive the order in interaction histories by a default prompting strategy. As a result, we aim to elicit the order-perceiving abilities of LLMs, by proposing two alternative prompting strategies and emphasizing the recently interacted items. Detailed descriptions of the proposed strategies have been given in Section 2.2. In Table 2, we can see that both recency-focused prompting and in-context learning can generally improve the ranking performance of LLMs, though the best strategy may vary on different datasets. The above results can be summarized as the following key observation:

Observation 1. LLMs *struggle to perceive the order* of the given sequential interaction histories. By employing specifically designed promptings, *LLMs can be triggered to perceive the order* of historical user behaviors, leading to improved ranking performance.

3.2 Do LLMs suffer from biases while ranking?

The biases and debiasing methods in conventional recommender systems have been widely studied [5]. For LLM-based recommendation models, both the input and output are natural language texts and will inevitably introduce new biases. In this section, we discuss two kinds of biases that LLM-based recommendation models suffer from. We also make discussions on how to alleviate these biases.

The order of candidates affects the ranking results of LLMs. For conventional ranking methods, the order of retrieved candidates usually will not



(a) Position Bias (b) Bootstrapping (c) Popularity Bias (d) Popularity of top-1 item *w.r.t.* $|\mathcal{H}|$

Fig. 3: Biases and debiasing methods in the ranking of LLMs. (a) The position of candidates in the prompts influences the ranking results. (b) Bootstrapping alleviates position bias. (c) LLMs tend to recommend popular items. (d) Focusing on historical interactions reduces popularity bias.

affect the ranking results [33,28]. However, for the LLM-based approach that is described in Section 2.2, the candidates are arranged in a sequential manner and infilled into a prompt. It has been shown that LLMs are generally sensitive to the order of examples in the prompts for NLP tasks [80,44]. As a result, we also conduct experiments to examine whether the order of candidates affects the ranking performance of LLMs. We follow the experimental settings adopted in Section 3.1. The only difference is that we control the order of these candidates in the prompts, by making the ground-truth items appear at a certain position. We vary the position of ground-truth items at $\{0, 5, 10, 15, 19\}$ and present the results in Figure 3(a). We can see that the performance varies when the ground-truth items appear at different positions. Especially, the ranking performance drops significantly when the ground-truth items appear at the last few positions. The results indicate that LLM-based rankers are affected by the order of candidates, *i.e., position bias*, which may *not* affect conventional recommendation models.

Alleviating position bias via bootstrapping. A simple strategy to alleviate position bias is to bootstrap the ranking process. We may rank the candidate set repeatedly for B times, with candidates randomly shuffled at each round. In this way, one candidate may appear in different positions. We then merge the results of each round to derive the final ranking. From Figure 3(b), we follow the setting in Section 3.1 and apply the bootstrapping strategy to *Ours*. Each candidate set will be ranked for 3 times. We can see that bootstrapping improves the ranking performance on both datasets.

Popularity degrees of candidates affect ranking results of LLMs. For popular items, the associated text may also appear frequently in the pre-training corpora of LLMs. For example, a best-selling book would be widely discussed on the Web. Thus, we aim to examine whether the ranking results are affected by the popularity of candidates. However, it is difficult to directly measure the popularity of item text. Here, we hypothesize that the text popularity can be indirectly